Cités des Métiers et des Compétences
La formation nouvelle génération

مكتب التكوين المهني و إنعاش الشغل
Office de la Formation Professionnelle et de la Promotion du Travail

Compte-rendu

Devoir N°2

Prénom : AYMEN

Nom : EL YAAKOUBI

Groupe : DEV106

INTRODUCTION :

Dans ce document, je présente les réponses du devoir de Python réalisées à l'aide de l'environnement Py.Charm. Ce travail m'a permis de mettre en pratique les notions fondamentales du langage Python, telles que les structures de contrôle, les fonctions et la manipulation des données. Les solutions proposées sont organisées de manière claire afin de faciliter la compréhension et la lecture.

Question 1:

```
class Personne:
    def __init__(self, reference, nom, prenom, age):
        self.__prenom = prenom
        self.__age = age
        self.__reference = reference
    def get_reference(self):
        return self.__reference
    def set_reference(self, reference):
        self.__reference = reference
    def get_nom(self):
        return self.__nom
    def set_nom(self, nom):
        self.__nom = nom
    def get_prenom(self):
        return self.__prenom
    def set_prenom(self, prenom):
        self.__prenom = prenom
    def get_age(self):
        return self.__age
    def set_age(self, age):
        self.__age = age
```

Figure1 : Déclaration de la classe Personne et des attributs privés

La classe Personne représente une personne dans l'entreprise. Elle contient des att ributs privés (référence, nom, prénom, âge) accessibles via des getters et setters, et un constructeur permet d'initialiser ces informations.

```
def __str__(self):
    return f"{self.__nom} {self.__prenom} {self.__age} {self.__reference}"
def __eq__(self, other):
    return self.__reference == other.__reference
@staticmethod
def ValiderReference(reference):
    ref_pattern = r'^[A-Z]{2}[0-9]{5}$'
```

Figure2 : Implémentation des méthodes ToString(), Equals() et ValiderReference

La classe redéfinit la méthode ToString() pour afficher toutes les informations du directeur. La méthode Equals() sert à comparer deux personnes : elles sont égales si elles ont la méme référence.Enfin, la méthode Valider référence() vérifie que la référence respecte un format précis : de lettres majuscules suivies de cinq chiffres.

Question 2:

```
class Directeur(Personne):
    def __init__(self, reference, nom, prenom, age, direction, fonction):
        super().__init__(reference, nom, prenom, age)
        self.__direction = direction
        self.__fonction = fonction
```

Figure3 : Création de la classe Directeur héritant de la classe Personne

Directeur : hérite de Personne. Elle récupère tous ses attributs/méthodes et ajoute les attributs spécifiques : direction et fonction.

```
def get_direction(self):
    return self.__direction
def set_direction(self, direction):
    self.__direction = direction
def get_fonction(self):
    return self.__fonction
def set_fonction(self, fonction):
    self.__fonction = fonction
def __str__(self):
    return super().__str__() + f" {self.__direction} {self.__fonction}"
```

Figure4 : Implémentation du constructeur et de la méthode ToString() dans la classe

Méthode ToString() dans Directeur redéfinie pour afficher les informations complètes du directeur.

Question 3:

```
class Employe(Personne):
    def __init__(self, reference, nom, prenom, age, tache, service):
        super().__init__(reference, nom, prenom, age)
        self.__tache = tache
        self.__service = service
```

Figure5 : Création de la classe Employe héritant de la classe Personne

Classe Employe (hérite de Personne)Attributs supplémentaires : tâche, service. Constructeur appelant celui de Personne via super().

```
@staticmethod
def valider_jour_mois_annee(date):
    pattern = r'^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/[0-9]{4}$'
    return bool(re.match(pattern, date))
def __str__(self):
    return super().__str__() + f" {self.__tache} {self.__service}"
```

Figure6 : Implémentation des méthodes valider_jour_mois_annee() et Tostring

· Attributs supplémentaires : tâche, service.· Constructeur appelant celui de Personne via super().· ToString() redéfinie pour afficher toutes les informations de l'employé.· valider_jour_mois_annee() ajoutée pour vérifier le format d'une date → jour/mois/année (exemple : 23/12/2024).

Question 4:

```
class Entreprise:
    def __init__(self, nom, adresse):
        self.__nom = nom
        self.__adresse = adresse
        self.__personnes = []
    def __str__(self):
        desc = f"Entreprise: {self.__nom}, adresse: {self.__adresse}\n"
        desc += "Liste de personnes:\n"
        for p in self.__personnes:
            desc += str(p) + "\n"
        return desc
```

Figure7 : Déclaration de la classe Entreprise et de ses attributs

Classe Entreprise permet de gérer l'ensemble des personnes appartenant à l'entreprise.

Attributs : nom, adresse, liste des données.

```
def ajouterPersonne(self, personne):
    self.__personnes.append(personne)

def chercherPersonne(self, reference):
    for p in self.__personnes:
        if p.get_reference() == reference:
            return p
    return None
def supprimePersonne(self, reference):
    p = self.chercherPersonne(reference)
    if p:
        self.__personnes.remove(p)
def listerDirecteurs(self):
    return [p for p in self.__personnes if isinstance(p, Directeur)]
def enregistrerPersonnes(self, fichier):
    with open(fichier, "w") as f:
        for p in self.__personnes:
            f.write(str(p) + "\n")
```

Figure8 : Implémentation des méthodes AjouterPersonne() et ChercherPersonne

Plusieurs méthodes sont implémentées pour gérer ces personnes :
Ajouter Personne() Permet d'ajouter une nouvelle personne à la liste.
Chercher Personne() Permet de rechercher une personne à partir de sa référence.
Liste Directeur() Permet d'afficher tous les directeurs de l'entreprise.
Supprimer personne Permet de supprimer une personne de la liste en utilisant sa référence
Enregistrer personne()Permet d'enregistrer la liste des personnes dans un fichier texte.

Question 8:Exucétion:

```
C:\Users\hp\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\hp\PyCharmMiscProject\.venv\devoir2.py
 des personn
Mona Samir 45 AB12346 Finance Manager
Ali Ahmed 25 AB12345
Sara Ali 30 CD67890 Comptabilité Finance
```

Figure10 :affichage de code

```
Validation de références
True
False
 Affichage
Entreprise: TechCorp, adresse: 123 Rue Centrale
Liste de personnes:
```

Figure10 :affichage de code

CONCLUSION :

En conclusion, ce travail pratique nous a permis de comprendre et d'appliquer les concepts fondamentaux de la programmation orientée objet. À travers la création des différentes classes telles que Personne, Directeur, Employé et Entreprise, nous avons mis en œuvre plusieurs notions importantes comme l'encapsulation, l'héritage, les constructeurs ainsi que la redéfinition des méthodes.